Pre-Gating Without Retraining:

Predicting every expert a token will need, from layer 0, with a bolt-on head

Parthasarathy Ramanujam

August 2026

Abstract

A node serving a mixture-of-experts model from partial storage has a timing problem. The router only names each layer's experts once computation reaches that layer, so the node discovers what it lacks one layer at a time, and stalls on the network at each discovery. We found the fix already sitting inside the model. Train a small head (12M parameters, backbone frozen) to read the hidden state after layer 0 and guess the expert choices of every later layer. On Qwen3-30B-A3B it gets 50.9 percent of all top-8 picks across 47 layers right; frequency guessing gets 22.7. Oddly, accuracy improves with depth. We wired the head into loom, an open distributed inference system, and measured a live network at two link speeds: 16 percent less aggregate wall time at 2.5 MB/s, 9 percent at 119 MB/s, output unchanged to the byte in both. The pair of points confirms what our trace replay predicts: the win is the share of runtime spent stalled on fetches, and it peaks where link and compute genuinely compete. Along the way we hit two negative results worth keeping. In our replay, looking one layer ahead was near useless even with perfect predictions. And on the model we measured, expert weights turn out to be full-rank matrices sharing almost nothing, which rules out compressing fetches by factoring them after the fact. Everything is released: traces, scripts, both heads, the serving code.

1 Introduction

The cost of partial serving is fetch bytes per token. Everything interesting hides one level down, in the shape of expert activations: how skewed usage is, whether consecutive tokens reuse experts, and whether early layers know what late layers will do. This paper measures that shape on two model families, works out how far systems tricks alone can go, and then goes after the one property that turned out to matter.

We contribute measurements first. Shape statistics for Qwen3-30B-A3B [1] and OLMoE-1B-7B [2] under a distributed cache, including an awkward fact: the model trained with the stronger load-balancing loss [3, 4] misses cache at 48.5 percent where the other misses at 19.6, same budget. In these two families at least, balance losses and caches pull in opposite directions. Second, a replay study. Plain LRU beat every policy we tried against it, shallow lookahead helped little at any depth we swept, and pre-gating (all layers predicted at layer 0) was the only scheme that changed the regime. Third, extraction: the frozen-backbone head, 69.9 percent on OLMoE and 50.9 on the 30B, still climbing with data. Fourth, deployment. One hook in the engine, one 66 MB file next to the model, and live-network batteries at two bandwidths: 16 percent off aggregate wall time on the slow link, 9 on the fast one, each explained by how much stall there was to hide. And fifth, the two negatives above, which we think save other groups real time.

System	Predicts from	Our difference
SiDA-MoE [5]	offline per-token predictor from embeddings	contextual layer-0 hidden state; modern top-8 decoders; WAN swarm costs, not one box
Pre-gated MoE [6]	trained into the architecture	most of the benefit, no retraining
MoE-Infinity [7]	sequence-level patterns	per-token, all layers at once, explicit lead-time accounting
Mixtral offloading [8]	next-layer heuristic plus LRU	we bound every lookahead depth with an oracle; depth 1 is the wrong regime
ExpertFlow [9]	a separate transformer predictor, all layers in one pass	no second model to run: our head reads the target's own layer-0 state, two matvecs; oracle-normalized accounting; deployed on a WAN swarm
EdgeMoE [10]	preloading heuristics, on-device	replay separates policy, depth, and pre-gate as independent axes

Table 1: Expert-prediction and offloading systems, and where this work differs.

2 Related work

One complementary line of work: AcceptMoE [11] reduces the same fetch bill from the opposite side, trimming the expert set a speculative-decoding verifier materializes. The two approaches compose.

3 The shape of expert activations

Property	Qwen3-30B-A3B (48L, 128E, top-8)	OLMoE-1B-7B (16L, 64E, top-8)
Coverage by hottest 25%	54.6%	37.5%
Adjacent-token expert reuse	45.1%	38.0%
Next-layer co-occurrence prediction	32.3% (base 9.9)	48.1% (base 17.4)
LRU miss at 25% budget	**19.6%**	48.5%
LRU vs static pin, MB per token	113 vs 338	93 vs 130

Table 2: Activation-shape statistics under a distributed cache, held-out halves throughout.

The table reduces to three facts, and the rest of the paper is built on them.

First, temporal locality is the resource. Adjacent tokens reuse roughly two of every five experts, and at every budget we swept that reuse was worth about three times more than static popularity: a cache tracking what just happened beats a pinned set of favorites. Section 4 confirms this the hard way, by trying to beat LRU and failing.

Second, popularity is fragile. With three prompt domains the hottest quarter of experts covered 63.4 percent of activations; with eleven domains, 54.6. Some of the skew that pinning schemes rely on is an artifact of narrow traffic, so pin-based serving looks best in exactly the benchmarks least like production.

Third, and most consequential here: usage statistics and routing structure are different properties, and training touches them differently. OLMoE routes much flatter than Qwen3 and pays double the miss rate for it, which we read as its stronger balance loss at work. Yet the flattening did not erase the structure underneath. OLMoE's layer-to-layer routing is more predictable than Qwen3's, not less. A training objective can ruin the statistics a cache needs while leaving intact the structure a predictor needs, and the

paper lives in that gap: Section 5 extracts the structure that survives, and Section 8 asks what it costs to repair the statistics.

4 The systems ceiling, by replay

Before training anything we wanted to know what caching and prefetching could do on their own. So: replay held-out traces through policies, report on the second half only.

Caching is a short story. LRU won. Decayed-frequency variants lost at every budget, global and per-layer slot pools tied, and recency-based prefetch did precisely nothing, because an LRU already contains whatever recency would have predicted.

Prefetching is a longer one. Give a depth-1 prefetcher perfect predictions and it still hides only a tenth of the stall; one layer of compute is a few milliseconds and a fetch is many. Pipelining deeper helps, peaking around depth 8 to 16 at roughly half the stall hidden. Pre-gating behaves differently. When layer 0 announces everything, layer L gets L layers of lead, and the oracle version takes a quarter-budget holder from 2.9 to 8.5 tok/s at 375 MB/s, and from 7.3 to 16.0 at 1250 against a compute floor of 20.8. A co-occurrence table gets you under a third of that improvement. Whether a learned head could close the rest became the obvious next question.

The replay model is deliberately crude: flat bandwidth, no per-fetch latency, prefetches bypass the cache. Section 6 is the reality check.

5 Extraction

The head is a two-layer MLP. Input: hidden state after layer 0. Output: one multi-hot over experts, per later layer. The backbone stays frozen, so the worst case is a useless file, never a worse model.

Model	Head	Baseline	Notes
OLMoE-1B-7B	**69.9%**	20.5%	29.5 → 48.3 → 60.0 → 69.9 as data and width grew; not saturated
Qwen3-30B-A3B	**50.9%**	22.7%	single pass, NF4 backbone; per-layer 39 to 65 percent

Table 3: All-layer top-8 prediction accuracy, held out.

We expected accuracy to decay with depth. It rises. Layer 10 of 16 is OLMoE's best-predicted layer at 69.2 percent, and layer 47 of 48 is Qwen3's at 65.4. Apparently the residual stream commits to its deep routing early, and commits hardest about the layers furthest away. Convenient, since those are also the layers with the most fetch lead time.

Two ablations pin the design down. Feeding the head raw token embeddings instead of layer-0 state (the static setting SiDA works in) still reaches 53.6 percent, well above the 20.5 baseline: token identity alone carries most of the routing signal, a point worth being fair about. Layer-0 state adds 6.4 points for zero lead-time cost; waiting one more layer adds 2.6 more but spends lead. Width scales monotonically too: 512 gives 53.6, 1024 gives 56.9, 2048 gives 60.0, 4096 gives 62.5, with no plateau in sight, so the accuracies in this paper are capacity floors, not ceilings.

Scored inside the replay, the OLMoE head captures 75 to 90 percent of what the oracle saves, with 2.4 times less wasted traffic than the table. One embarrassment worth recording: our first prediction dump was written during training, so it scored the half-trained head, and the sim cheerfully showed a "60 percent" predictor losing to the dumb table until we redumped from the final weights. Evaluate the model you shipped, not the one mid-descent.

6 Deployment

Integration was small by design. The engine already computes the layer-0 hidden state; the head is two matvecs on kernels the engine already has; the async fetch pool already existed for a one-layer lookahead scheme [12]. One hook connects them, nearest layer first so the tightest deadline leads the queue. A dropped prediction costs nothing. A wrong one still warms the store. With the flag off, or the head missing, nothing changes at all, and output is byte-identical either way because routing never reads a prediction.

The measurement: a partial holder of Qwen3-30B on the live devnet [13]. We asked for 50 percent; measuring revealed the sync path ignores the flag and every battery ran at 70.6 percent holdings (filed as loom issue 252, disclosed here because the numbers depend on it). Greedy decoding keeps every run on the identical 96 tokens, and the store restores from a snapshot before each run because fetches persist. The first battery ran over a 2.5 MB/s link, prefetch's worst case: it cannot create bandwidth there, only overlap.

Pair	Baseline	Pre-gated	Saving
1	2682 s	1949 s	27.3%
2	2361 s	2334 s	1.2%
3	2419 s	1981 s	18.1%
aggregate	7462 s	6264 s	**16.1%**

Table 4: Slow-link battery (2.5 MB/s), snapshot-controlled pairs.

The spread deserves a sentence. On a saturated link the fetched bytes are fixed, so the head's only edge is filling the round-trip gaps that fetch-on-miss leaves idle, and how many such gaps a baseline run has depends on the peer's load at that moment. Two pairs saved 18 to 27 percent. One saved almost nothing. The aggregate is 16 percent in the one regime where prefetch has no bandwidth to work with.

The second battery ran the same protocol from a Hetzner Helsinki node with a measured 119 MB/s to the peer, five pairs: baselines 277 to 286 seconds, pre-gated 252 to 261, aggregate saving 9.0 percent with tight spread, and a replication battery landed at 6.4. Lower than the slow link, and the decomposition says why: at 70.6 percent holdings on four shared vCPUs the fast link makes fetches nearly free relative to compute, so little stall remains to hide. The win tracks the stall share, not the bandwidth. The regime the replay predicts big numbers for, fast link with low holdings, is exactly the configuration issue 252 currently prevents building; that point lands when the fix does.

Prefetch budget, from the replay with the head's ranked predictions: $k = 8$ reaches 19.9 tok/s at 1250 MB/s, $k = 12$ reaches 22.1, $k = 16$ reaches 23.5 against the oracle's 27.0, at 4x the wasted bytes of $k = 8$. Over-fetching is a real knob: it buys most of the remaining oracle gap when bandwidth is cheap.

7 What cannot be retrofitted

We wanted fetches to shrink from megabytes to kilobytes. The idea: factor each layer's experts into a shared core plus low-rank per-expert deltas, ship the core with the model, fetch only deltas, heal the damage by distillation. Cheap to test, so we tested before building. The answer is no. On Qwen3-30B the cross-expert mean holds about 1 percent of the energy; there is no shared core. The residual spectra are close to flat. Rank 32 of 768 keeps 16 to 19 percent of the energy, rank 64 stays under 30, and no distillation recovers a signal that was mostly discarded. Fine-grained fetch structure has to be imposed at

pretraining time. We suspect the same force is at work here as in Section 3: balance-loss training pushes experts toward genuinely independent, full-rank matrices.

8 The training-side exchange rate

Extraction solves prediction but does not touch the bytes, and Section 3 says the bytes are set by how routing was trained. So the last experiment retrains just the routers of OLMoE (2M parameters, backbone frozen): balance loss off, a symmetric-KL consistency term between adjacent tokens on, swept at three weights, 8M tokens each [14]. A frozen backbone makes any perplexity change the honest price of the routing change. Same trace scripts, same token counts, same metrics as everything above.

Config	ppl	Coverage@25%	Stickiness	LRU@25% miss	Fetch MB/tok
frozen	9.73	37.5%	38.0%	48.5%	93.2
$\lambda = 0$ (control)	10.64	40.6%	35.2%	51.6%	99.2
$\lambda = 0.5$	11.03	44.1%	46.0%	40.2%	77.1
$\lambda = 2.0$	12.66	53.6%	**57.4%**	**26.9%**	**51.7**

Table 5: Router-only retraining: consistency weight against quality and cacheability.

The control earns its keep: removing the balance loss and letting routers drift makes the shape slightly worse on every cache metric while still costing 9 percent perplexity. The consistency term does all the useful work. At $\lambda = 2.0$ the miss rate nearly halves and fetch traffic drops 45 percent, for a steep 30 percent perplexity price at this small training budget. Two observations we want to test at larger scale: the price may shrink with longer healing, since 8M tokens is barely a nudge for a 7B model, and $\lambda = 2.0$'s statistics land almost exactly on Qwen3's natural shape (53.6 versus 54.6 percent coverage), as if the consistency pressure recovers what the stronger balance loss had erased. Whether that is coincidence or mechanism is a good question for the camera-ready.

These effects stack with the head, in principle: retraining cuts the bytes, prediction hides what remains. Measuring the combination end to end is future work.

9 Discussion

What the numbers support: reactive caching needs nothing beyond LRU in our sweeps, and shallow lookahead is bounded hard by lead time under our replay assumptions, oracle or not. Whole-token prediction was sitting inside unmodified checkpoints, readable by a head smaller than one expert. Byte savings need retrained routers, or a pretrain designed for distribution from day one, borrowing what already exists: DeepSeekMoE's shared experts [15, 16] absorb the common computation, and PEER's tiny-expert granularity [17] would shrink each fetch. A model built that way would stack every effect in this paper. The bolt-on head needed none of it, which is why it shipped first, and why the flag can default on.

10 Limitations

Stated plainly. The deployment batteries use one greedy prompt trajectory; the eleven-domain battery is queued. Every battery ran at 70.6 percent holdings because the sync bug we filed (loom issue 252) prevents lower, so the fetch-heaviest regime is simulated but not yet measured. The replay's stall model is flat-bandwidth with no per-fetch latency term. The ablations ran on OLMoE; we assume their trends

transfer to the 30B and have verified only the headline numbers there. And the exchange-rate table is an 8M-token result on a 7B model, far from converged, which is why we report it as a rate and not a verdict.

Artifacts. Both trained heads, the LPG1 export, the trace files, and all analysis and training scripts ship in the loom repository [13] under Apache 2.0, alongside Mixtral-class open weights we build on [1, 2, 18].

References

[1] An Yang, Anfeng Li, et al. Qwen3 technical report, 2025.

[2] Niklas Muennighoff, Luca Soldaini, et al. Olmoe: Open mixture-of-experts language models, 2024.

[3] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity, 2021.

[4] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding, 2020.

[5] Zhixu Du, Shiyu Li, Yuhao Wu, Xiangyu Jiang, Jingwei Sun, Qilin Zheng, Yongkai Wu, Ang Li, Hai Li, and Yiran Chen. Sida-moe: Sparsity-inspired data-aware serving for efficient and scalable large mixture-of-experts models, 2024. MLSys 2024.

[6] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. Pre-gated moe: An algorithm-system co-design for fast and scalable mixture-of-expert inference, 2024. ISCA 2024.

[7] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. Moe-infinity: Efficient moe inference on personal machines with sparsity-aware expert cache, 2024.

[8] Artyom Eliseev and Denis Mazur. Fast inference of mixture-of-experts language models with offloading, 2023.

[9] Xin He, Shunkang Zhang, Yuxin Wang, Haiyan Yin, Zihao Zeng, Shaohuai Shi, Zhenheng Tang, Xiaowen Chu, Ivor Tsang, and Yew Soon Ong. Expertflow: Efficient mixture-of-experts inference via predictive expert caching and token scheduling, 2024.

[10] Rongjie Yi, Liwei Guo, Shiyun Wei, Ao Zhou, Shangguang Wang, and Mengwei Xu. Edgemoe: Empowering sparse large language models on mobile devices, 2023.

[11] Shuang Liang, Hao Mark Chen, et al. Acceptmoe: Commitment-weighted self-sizing verifier expert sets for efficient moe speculative decoding, 2026.

[12] JustVugg. colibri: Cpu expert-streaming inference for large moe models. `https://github.com/JustVugg/colibri`, 2026.

[13] Parthasarathy Ramanujam. loom: a distributed expert cache for large moe inference. `https://github.com/ch4r10t33r/loom`, 2026.

[14] Guilherme Penedo, Hynek Kydlíček, et al. The fineweb datasets: Decanting the web for the finest text data at scale, 2024.

[15] Damai Dai, Chengqi Deng, et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models, 2024.

[16] DeepSeek-AI. Deepseek-v3 technical report, 2024.

[17] Xu Owen He. Mixture of a million experts, 2024.

[18] Albert Q. Jiang, Alexandre Sablayrolles, et al. Mixtral of experts, 2024.